



Guia da equipe de infraestrutura: desenvolvimento de xApps

Acesso seguro, Git, Nephio, Flux, Near-RT RIC e validação KPM

OBJETIVO

Levar um integrante do primeiro SSH até a entrega revisada de uma xApp, sem conceder administração total do cluster compartilhado.

Ambiente acadêmico e POC - não é procedimento de produção

Este guia permite que um novo integrante valide o caminho completo de uma xApp sem receber administração total do cluster. O objetivo inicial é desenvolver, observar e entregar a mudança pelo Git. A promoção para a RAN compartilhada continua exigindo revisão de outro membro da infraestrutura.

1. O mapa mental antes dos comandos

Não escolha entre Kubernetes e Nephio: eles fazem trabalhos diferentes.

```
código da xApp -> GitHub Actions -> imagem amd64/arm64
  -> blueprint/variante Nephio -> Git de deployment -> Flux
  -> Deployment Kubernetes -> Near-RT RIC -> E2 -> O-DU simulado
  -> métricas Prometheus -> Grafana
```

- Kubernetes executa pods, redes, Services e volumes.
- Near-RT RIC recebe telemetria E2 da RAN e entrega indicações KPM para xApps.
- A xApp implementa o algoritmo do pesquisador. Ela não cadastra SIM no Open5GS.
- Nephio especializa e publica pacotes de configuração por laboratório.
- Flux aplica no cluster o estado publicado no Git.
- Prometheus armazena métricas; Grafana mostra os resultados.

2. Onde você entra e o que pode fazer

A VM 102 é o ponto de entrada dos integrantes. Ela fica em `192.168.71.100`, acessível somente pela VPN NMI. A máquina também executa um DU/K3s legado: não pare serviços, não remova containers e não use `sudo kubectl`.

Cada integrante possui uma conta Linux e uma identidade Kubernetes individual.

Ação	Permitida?
Listar nós, pods, Services e Deployments	Sim
Ler logs da pilha e objetos Nephio/Porch	Sim
Criar recursos no namespace <code>student-lab</code>	Sim
Ler Kubernetes Secrets	Não
Alterar <code>ran</code> , <code>5g-core</code> , <code>near-rt-ric</code> ou <code>ricxapp</code>	Não
Publicar uma xApp na RAN sem revisão	Não

Essa separação não impede o desenvolvimento. Ela evita que um erro no primeiro experimento derrube a bancada usada por toda a equipe.

3. Primeiro acesso

Conecte a VPN NMI e entre com a conta informada pelo responsável:

```
ssh SEU_USUARIO@192.168.71.100
```

No primeiro acesso, troque a senha temporária quando o SSH solicitar. Depois, valide sua identidade e o contexto:

```
whoami
kubectl auth whoami
kubectl config current-context
kubectl config view --minify
```

O usuário mostrado por `kubectl auth whoami` deve ser o mesmo do Linux, o grupo deve conter `openran-students` e o contexto deve ser `student-lab`.

Teste as fronteiras de permissão:

```
kubectl auth can-i list pods --all-namespaces
kubectl auth can-i create deployments.apps -n student-lab
kubectl auth can-i get secrets --all-namespaces
kubectl auth can-i patch deployments.apps -n ran
```

O resultado esperado é **yes, yes, no, no**. Use sempre `kubectl`, nunca `sudo kubectl`: o root da VM 102 pode selecionar o K3s legado, que é outro contexto e não faz parte do exercício.

4. Reconhecer a bancada sem modifica-la

```
kubectl get nodes -o wide
kubectl -n ran get pods -o wide
kubectl -n near-rt-ric get pods -o wide
kubectl -n ricxapp get deploy,pods,services -o wide
kubectl -n 5g-core get pods
kubectl get packagevariants -A
kubectl -n flux-system get kustomizations
```

Todos os cinco nós NMI devem aparecer **Ready**. Os pods principais devem estar **Running** ou **Completed**, sem reinícios crescendo continuamente. Falha da pilha base deve ser registrada e comunicada antes de alterar o código da xApp.

Para ver a xApp de referencia:

```
kubectl -n ricxapp logs deployment/r4-simple-mon -c xapp --tail=80
kubectl -n ricxapp get service -l app=r4-simple-mon
```

KPI significa indicador-chave de desempenho. A `r4-simple-mon` assina `DRB.UETHPD1`, recebe indicações E2SM-KPM e exporta throughput downlink. O Report Style 1 atual agrega a medida no O-DU; ele prova carga de radio, mas não atribui o valor a um IMSI específico.

5. Preparar uma copia de trabalho

Trabalhe na sua casa, não dentro de diretórios do sistema. O diretório `/dados/openran-infra` existe somente para arquivos compartilhados da equipe.

```
cd ~
git clone https://github.com/lucasrodri/oran-stack.git
cd oran-stack
git switch main
git pull --ff-only
git switch -c lab/NOME-DA-XAPP
```

Escolha um nome DNS em minúsculas, com hifens e sem espaços. Cada pessoa usa sua própria branch; não desenvolva diretamente em `main`.

6. Gerar e programar uma xApp KPM

Exemplo com uma xApp chamada `kpm-team-lab`, métrica `DRB.UETHPD1` e período de 2,5 segundos:

```
./scripts/new-kpm-xapp.sh kpm-team-lab DRB.UETHPD1 2500
```

O gerador cria o código Python, o Team Blueprint, a variante NMI e os objetos de entrega Flux. O ponto principal de programação é:

```
xapps/python/kpm_team_lab_xapp.py -> indication_callback(...)
```

Comece com comportamento observável e sem controle da RAN: média móvel, contador, limiar ou classificação `idle/active/busy`. O callback não deve bloquear e os logs não devem imprimir uma linha para cada indicação.

Valide antes do primeiro commit:

```
python3 -m py_compile xapps/python/kpm_team_lab_xapp.py
grep -R 'REPLACE_WITH_MULTIARCH_IMAGE_DIGEST' \
  packages/nephio/kpm-team-lab \
  infra/nephio/blueprints/kpm-team-lab-nmi-variant.yaml
git diff --check
git status --short
```

7. Entregar pelo Git e gerar a imagem

```
git add xapps/python/kpm_team_lab_xapp.py \
  packages/nephio/kpm-team-lab \
  infra/nephio/blueprints/kpm-team-lab-nmi-variant.yaml \
  infra/nephio/nmi-onboarding/kpm-team-lab-rbac.yaml \
  infra/nephio/nmi-onboarding/kpm-team-lab-flux-sync.yaml
git commit -m "feat(xapp): add KPM team laboratory monitor"
git push -u origin lab/NOME-DA-XAPP
```

O workflow `Build multi-architecture laboratory xApps` deve construir `linux/amd64` e `linux/arm64`. Depois do build, copie o digest do índice multiarch, no formato `sha256:<64 caracteres>`, e substitua `REPLACE_WITH_MULTIARCH_IMAGE_DIGEST` nos dois arquivos indicados pelo gerador. Faça outro commit e abra um pull request.

Nunca use `:latest`: a variante Nephio precisa apontar para uma imagem imutável por digest para permitir repetição e rollback verificável.

8. Promoção controlada com Nephio

O integrante entrega no pull request:

- código e teste da xApp;
- imagem multiarch fixada por digest;
- Team Blueprint;
- PackageVariant NMI;
- RBAC e sincronização Flux da nova xApp;
- nome da métrica, período KPM e critério de aceite.

Outro membro da infraestrutura revisa e executa a promoção administrativa:

```
Draft -> Proposed -> Published -> Git NMI -> Flux Ready -> Pod ricxapp
```

Enquanto isso, o autor acompanha sem privilégio administrativo:

```
kubect1 get packagevariants -A
kubect1 -n flux-system get kustomizations
kubect1 -n ricxapp get deployment,pods,services -l app=kpm-team-lab
kubect1 -n ricxapp logs deployment/kpm-team-lab -c xapp --tail=100
```

O Nephio publica configuração; ele não executa a xApp. O Flux reconcilia o Git e o Kubernetes executa o pod. Essa separação é a parte central do experimento.

9. Aceite funcional e observação

O responsável pela bancada dispara tráfego limitado pelo UE simulado. O autor acompanha a xApp e o Grafana. O aceite exige:

- Deployment disponível e pod `Ready`;
- xApp registrada no AppMgr;
- uma assinatura KPM ativa;
- contadores RMR/KPM aumentando;
- `DRB.UEThpD1` maior que zero durante o tráfego;
- retorno ao estado ocioso depois do teste;
- nenhum restart crescente do pod.

Links pela VPN NMI

Interface	Endereço
Grafana - visao geral	http://192.168.72.10:30300/d/oran-overview/o-ran-stack-overview
Grafana - logs	http://192.168.72.10:30300/d/oran-logs/o-ran-kubernetes-logs
Grafana - CIC ARM	http://192.168.72.10:30300/d/oran-multisite/cic-arm-lab
Open5GS WebUI	http://192.168.72.10:30454
Nephio WebUI	http://192.168.71.30:30707/config-as-data
Alertmanager	http://192.168.72.10:30301

10. Diagnóstico sem enxugar gelo

Siga a ordem causal. Não reinicie componentes aleatoriamente.

1. `kubectl get nodes`: o cluster está acessível e os nós estão *Ready*?
2. `kubectl -n ran get pods`: CU, DU e UE estão saudáveis?
3. `kubectl -n near-rt-ric get pods`: E2Term, E2Mgr e SubMgr estão saudáveis?
4. `kubectl -n ricxapp get pods`: a imagem iniciou e o pod está *Ready*?
5. Logs da xApp: houve registro, assinatura e indicações RMR?
6. Grafana: a série existe antes, durante e depois do tráfego?

Sintoma	Primeira verificacao
SSH não conecta	VPN NMI, IP e usuário
<code>Unauthorized</code> no <code>kubectl</code>	<code>~/.kube/config</code> e <code>kubectl auth whoami</code>
<code>Forbidden</code> em <code>student-lab</code>	namespace e verbo em <code>kubectl auth can-i</code>
<code>Forbidden</code> em RAN/RIC	comportamento esperado; abra pedido de promoção
<code>ImagePullBackOff</code>	digest multiarch e acesso ao GHCR
xApp Ready sem KPM	conexão E2, assinatura, RMR e tráfego do UE
Grafana sem série	endpoint <code>/metrics</code> , Service e descoberta Prometheus

Registre o horário, comando, saída curta e componente afetado. Logs enormes sem janela de tempo dificultam o diagnóstico e não devem ser guardados nesta POC.

11. Definição de concluído

O primeiro exercício está concluído quando o integrante consegue:

1. entrar pela VPN e VM 102;
2. explicar Kubernetes, RIC, xApp, Nephio, Flux e Grafana em uma frase cada;
3. observar a pilha sem usar `sudo kubectl`;
4. gerar uma xApp com nome único;
5. implementar e validar uma mudança pequena;

-
6. publicar branch e imagem multiarch por digest;
 7. abrir o pull request com critério de aceite;
 8. acompanhar a promoção Nephio/Flux;
 9. provar KPM no Grafana durante tráfego do UE;
 10. documentar resultado e rollback.

Para aprofundar, consulte [docs/STUDENT_XAPP_LAB.md](#), [docs/NEPHIO_MANAGEMENT_CLUSTER.md](#) e [docs/NMI_DEPLOYMENT.md](#) no repositório.